

دانشگاه صنعتی امیرکبیر (پلی‌تکنیک تهران)

دانشکده مهندسی کامپیوتر

درس اصول طراحی نرم‌افزار — نیم‌سال اول ۱۴۰۵–۱۴۰۶

سند جامع مشخصات مدل دامنه

فروشگاه آنلاین شیءگرا

Object-Oriented Domain Model Specification

نسخه نهایی یکپارچه‌شده بر مبنای اصل اتحاد دیاگرام و پیاده‌سازی (Reconciled Baseline)

موضوع فاز دوم:	مدل‌سازی دقیق دامنه، روابط کلاس‌ها، قوانین کسب‌وکار و قراردادها
فناوری هدف:	جاوا ۲۱ (Java 21 LTS) با رکوردهای تغییرناپذیر
معماری حاکم:	معماری چندلایه‌ای ناب دامنه-محور (Domain-Driven Layered)
وضعیت مدل:	مصوب و منطبق بر اصل اتحاد متدها و مشخصات پیاده‌سازی
تاریخ نگارش:	آبان ۱۴۰۵ / نوامبر ۲۰۲۶

تهیه‌شده بر مبنای محتوای آموزشی اسلایدها و جزوات درسی استاد مهدی مزینانی

فهرست مطالب

۱ شناسنامه و رویکرد معماری مدل (Architectural Overview) ۲
۱.۱ اصول بنیادین حاکم بر مدل‌سازی ۲
۲.۱ معماری لایه‌ای و تفکیک مسئولیت‌ها ۲
۳.۱ قوانین و رفتارهای حاکم بر دامنه (Core Domain Rules & Policies) ۳

۲ ماتریس ردیابی جامع ۳۶ داستان کاربری و ۱۲ داستان دامنه (Traceability Matrix) ۴

۳ تفکیک مفاهیم دامنه: موجودیت‌ها در برابر اشیاء مقداری (Entities vs. Value Objects) ۴
۱.۳ موجودیت‌ها (Entities) ۵
۲.۳ اشیاء مقداری (Value Objects) ۵

۴ کاتالوگ جامع کلاس‌های لایه مدل (Domain Model Catalog) ۶
۱.۴ کلاس Quantity (Value Object — Record) ۶
۲.۴ کلاس Money (Value Object — Record) ۶
۳.۴ کلاس Address (Value Object — Record) ۷
۴.۴ کلاس User (Entity) ۸
۵.۴ کلاس Product (Entity) ۸
۶.۴ کلاس CartItem (Value Object — Record) ۹
۷.۴ کلاس Cart (Entity / Aggregate Root) ۱۰
۸.۴ کلاس OrderItem (Value Object — Record) ۱۰
۹.۴ کلاس Order (Entity / Aggregate Root) ۱۰
۱۰.۴ کلاس PaymentTransaction (Value Object — Record) ۱۱
۱۱.۴ کلاس Category (Value Object / Entity) ۱۲
۱۲.۴ کلاس Coupon (Value Object — Record) ۱۲
۱۳.۴ کلاس Review (Value Object — Record) ۱۲
۱۴.۴ کلاس ContactInfo (Value Object — Record) ۱۲
۱۵.۴ انواع شمارشی مدل دامنه (Domain Enumerations) ۱۳

۵ کاتالوگ جامع کلاس‌های لایه سرویس (Application & Domain Services) ۱۴
۱.۵ AuthService ۱۴
۲.۵ UserService ۱۵
۳.۵ ProductService ۱۵
۴.۵ CartService ۱۶
۵.۵ OrderService ۱۶
۶.۵ ReviewService ۱۷

۶ طراحی معماری لایه Presentation (CLI Layer Specification) ۱۸
۱.۶ اجزای اصلی لایه CLI ۱۸

۷ مشخصات کلاس راه‌انداز اصلی (Main.java) و ابزارهای پشتیبان ۱۹

۸ نمودارهای جامع ساختاری و روابط (UML Class Diagrams) ۲۰
۱.۸ جدول جامع ارتباط‌ها و تعداد آن‌ها (Associations & Multiplicities) ۲۰
۲.۸ نمودار روابط کلاس‌های لایه Model (Core Domain Model) ۲۰
۳.۸ نمودار ماشین وضعیت چرخه عمر سفارش (Order Lifecycle Statechart) ۲۱

۹ اعتبارسنجی جامع مدل و ارزیابی ده‌گانه (Validation & Compliance) ۲۳

۱۰ ثبت وضعیت و آمادگی برای ورود به فاز بعدی ۲۳

۱ شناسنامه و رویکرد معماری مدل (Architectural Overview)

این مستند، به عنوان منبع موثق و خط مبنای واحد (Single Source of Truth)، مشخصات کامل معماری، قوانین کسب‌وکار و مدل دامنه سامانه فروشگاه آنلاین را تبیین می‌نماید. مدل حاضر بر اساس نیازمندی‌های فاز دوم پروژه و با وفاداری به آموزه‌های چهار جزوه درسی تدوین شده است:

- `clean-code.pdf`: رعایت اسامی بامعنا، متدهای تک‌منظوره و کوچک، تفکیک پرسش از فرمان (CQS) و کاهش وابستگی‌ها.
- `dataAbstraction.pdf`: تفکیک دقیق انواع جهش‌پذیر و تغییرناپذیر، نامتغیرهای بازنمایی (RI) و توابع انتزاع (AF) و پیشگیری از افشای بازنمایی.
- `java-code-conventions.pdf`: رعایت استانداردها و قراردادهای رسمی کدنویسی جاوا.
- `procedural-abstraction.pdf`: تدوین دقیق پیش‌شرط‌ها (REQUIRES)، اثرات جانبی (MODIFIES) و نتایج تضمین‌شده (EFFECTS).

۱.۱ اصول بنیادین حاکم بر مدل‌سازی

۱. **معماری چندلایه‌ای ناب (Layered Architecture):** تفکیک قطعی لایه‌های واسط کاربری متنی (CLI)، لایه سرویس‌های کاربردی (Service) و لایه مدل کسب‌وکار (Model). لایه مدل کاملاً مستقل بوده و هیچ وابستگی به لایه‌های بالاتر ندارد.

۲. **استفاده از قابلیت‌های نوین جاوا ۲۱:** مدل‌سازی کلیه اشیاء مقداری تغییرناپذیر به صورت رکوردهای جاوا (`record`) با سازنده‌های فشرده (Compact Constructors).

۳. **مشخصه‌سازی علمی عملیات:** تدوین قرارداد سه‌بخشی `REQUIRES/MODIFIES/EFFECTS` برای تمامی متدهای حساس لایه‌های مدل و سرویس.

۴. **اصل اتحاد و یکپارچگی مدل (Union Baseline):** تمامی عناصر ساختاری و رفتاری موجود در کاتالوگ، دیاگرام‌های UML و کدهای پیاده‌سازی‌شده با یکدیگر هماهنگ شده‌اند.

۲.۱ معماری لایه‌ای و تفکیک مسئولیت‌ها

سامانه به چهار بخش اصلی تفکیک شده است که جریان وابستگی صرفاً از لایه‌های بالاتر به لایه‌های پایین‌تر مجاز است:

بخش / لایه	پکیج مربوطه	مسئولیت اصلی	وابستگی‌های مجاز
Model	`org.example.model`	داده‌ها، اشیاء مقداری، موجودیت‌ها، نامتغیرها و رفتارهای داخلی اشیاء.	کتابخانه استاندارد جاوا
Service	`org.example.service`	اجرای سناریوهای کاربردی، احراز هویت، کاتالوگ، سبد، پرداخت، سفارش، نظرات و مخازن داده درون‌حافظه‌ای.	Model، استثناها و ابزارهای عمومی
CLI	`org.example.cli`	دریافت ورودی‌ها، اعتبارسنجی مرزی، فراخوانی سرویس‌ها و نمایش منوها.	Model و Service
Main & Util	`org.example`	ساخت مخازن داده مشترک، اتصال اجزا (DI)، ابزار هشینگ و راه‌اندازی.	تمامی اجزای لازم برای اجرا

جدول ۱: تفکیک مسئولیت‌ها و وابستگی‌های مجاز در معماری سامانه

۳.۱ قوانین و رفتارهای حاکم بر دامنه (Core Domain Rules & Policies)

۱. **هویت و یکتایی در ثبت‌نام:** ثبت‌نام حداقل به ایمیل یا موبایل نیاز دارد. هر مشخصه واردشده باید معتبر و غیرتکراری باشد. شیء مقداری `ContactInfo` فرمت RFC ایمیل و شماره همراه ایران (`09[0-9]{9}`) را اعتبارسنجی می‌کند؛ سرویس‌های `AuthService` و `UserService` عدم تکراری بودن را تضمین می‌کنند.

۲. **ورود و ابطال نشست:** ورود با ایمیل یا موبایل انجام می‌گیرد. رمز عبور به صورت هش امن نگهداری شده و هرگز در پروفایل فاش نمی‌شود. خروج کاربر، توکن نشست جاری را باطل می‌کند. بازیابی رمز، کلیه نشست‌های قبلی کاربر را منقضی می‌سازد.

۳. **شبیه‌سازی بازیابی رمز عبور:** در محیط متنی، کد ۶ رقمی موقت کنسولی با اعتبار ۱۰ دقیقه تولید می‌شود و پس از یک‌بار مصرف ابطال می‌گردد.

۴. **رؤیت‌پذیری کاتالوگ:** تنها محصول فعال در دسته فعال به خریدار نمایش داده می‌شود. اگر محصولی پس از افزودن به سبد غیرفعال شود، در مرحله تسویه‌حساب یا پرداخت متوقف می‌شود.

۵. **مثبت بودن تعداد و عدم تجاوز از موجودی:** تعداد اقلام همواره اکیداً مثبت ($Quantity > 0$) بوده و از موجودی انبار بیشتر نمی‌شود. افزودن مجدد کالا، تعداد همان سطر سبد را افزایش می‌دهد.

۶. **محدودیت‌های متنی و ارقامی:** طول جستجوی کالا بین ۳ تا ۱۰۰ کاراکتر، طول نظر کالا حداکثر ۲۰۰۰ کاراکتر، امتیاز ۱ تا ۵ ستاره، کد پستی دقیقاً ۱۰ رقمی و تلفن همراه منطبق بر الگوی معتبر است.

۷. **محاسبات مالی و کوپن:** قیمت نهایی سفارش برابر است با مجموع قیمت اقلام منهای تخفیف کوپن به علاوه هزینه ارسال (فعلاً صفر). کوپن باید منقضی نشده باشد و شرط حداقل خرید را احراز کند.

۸. **پرداخت و کسر اتمیک موجودی:** پرداخت فقط برای سفارش‌های در وضعیت `PENDING` یا `PAYMENT_FAILED` مجاز است. پیش از کسر موجودی، کفایت موجودی کل اقلام به صورت اتمیک بررسی می‌شود. شکست پرداخت موجودی را تغییر نمی‌دهد و امکان تلاش مجدد محفوظ است.

۹. **تسویه جزئی سبد خرید:** هنگام پرداخت موفق، صرفاً اقلامی از سبد خرید حذف می‌شوند که در سفارش قطعی شده‌اند و از زمان تسویه تا پرداخت بدون تغییر مانده‌اند. اقلام جدید یا ویرایش‌شده در سبد حفظ می‌شوند.

۱۰. **استرداد موجودی در لغو سفارش:** لغو سفارش پرداخت‌شده منجر به بازگشت قطعی موجودی کالاها به انبار می‌گردد. لغو مجدد سفارش لغوشده ممنوع است.

۱۱. **انزوای داده‌های مشتری:** مشتری فقط به جزئیات، پرداخت و لغو سفارش‌های متعلق به شناسه کاربری خود دسترسی دارد (متدهای دارای `userId`).

۱۲. **کنترل دسترسی نقش‌محور (RBAC):** صرفاً نقش `ADMIN` مجاز به ورود به پنل مدیریت است. نقش `STOREKEEPER` در مدل شمارشی جهت سازگاری تعریف شده اما دسترسی مدیریتی ندارد.

۱۳. **تصاویر کالا:** تصاویر به صورت URLهای معتبر اینترنتی ذخیره شده و در کنسول نمایش داده می‌شوند و توسط مدیر قابل ویرایش هستند.

۱۴. **امتیاز محبوبیت و جدیدترین:** مرتب‌سازی جدیدترین بر مبنای تاریخ ثبت (`createdAt`) و محبوب‌ترین بر اساس شمارنده فروش تجمعی (`soldCount`) در پرداخت‌های موفق انجام می‌گیرد.

۲ ماتریس ردیابی جامع ۳۶ داستان کاربری و ۱۲ داستان دامنه (Traceability Matrix)

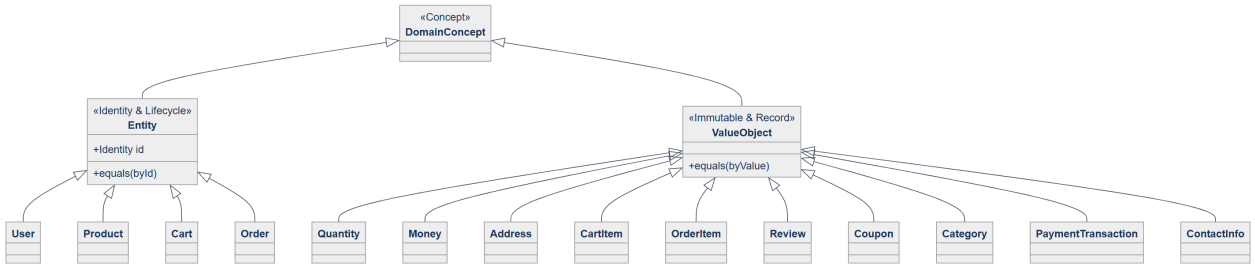
برای اثبات پوشش ۱۰۰٪ نیازمندی‌ها، ماتریس جدول ۲ نحوه اتصال داستان‌های کاربری (US) و داستان‌های دامنه (DS) را به کلاس‌ها و متدهای مدل مشخص می‌سازد:

حوزه کارکردی	داستان کاربری	داستان دامنه	کلاس‌های لایه Model	متدهای هماهنگ‌کننده لایه Service
ثبت‌نام و ورود	US 01, 02, 03	DS 01, 02	User, UserRole, ContactInfo	AuthService.register, login, logout
بازیابی رمز	US 04	DS 03	User, PasswordReset	AuthService.requestPasswordReset, resetPassword
پروفایل و آدرس	US 05, 06, 07	DS 04	User, Address, ContactInfo	UserService.getProfile, updateProfile, addAddress, updateAddress, removeAddress, setDefaultAddress
کاتالوگ کالا	US 08–14	DS 05	Product, Category, Money, Quantity	ProductService.getAllActiveProducts, getProductById, searchProducts, filterAndSort
سبد خرید	US 15–19	DS 06	Cart, CartItem, Quantity, Product	CartService.getCart, addItem, updateItemQuantity, removeItem, removeCheckedOutItems, clearCart
تسویه‌حساب	US 20–23	DS 07	Order, OrderItem, Coupon, Address	OrderService.checkout
پرداخت و تراکنش	US 24, 25	DS 07, 08	Order, PaymentTransaction, PaymentMethod	OrderService.processPayment
تاریخچه سفارش	US 26, 27, 28	DS 09	Order, OrderItem, OrderStatus	OrderService.getOrderHistory, getOrderDetails, cancelOrder
نظرات و امتیاز	US 30, 31, 32	DS 10	Review, Product	ReviewService.addReview, getProductReviews
مدیریت کاتالوگ	US 33, 34, 35	DS 11	Product, Category, Money	ProductService.addProduct, updateProduct, setProductImages, deactivateProduct
انبارداری و پردازش	US 29, 36	DS 12	Product, Order, OrderStatus, Quantity	ProductService.adjustStock; OrderService.getAllOrders, getOrdersByStatus, updateOrderStatus

جدول ۲: ماتریس ردیابی جامع داستان‌های کاربری و دامنه به اجزای مدل و سرویس

۳ تفکیک مفاهیم دامنه: موجودیت‌ها در برابر اشیاء مقداری (Entities vs. Value Objects)

بر اساس آموزه‌های جزوه `dataAbstraction.pdf`، تفکیک مفاهیم بر پایه اصول شیءگرایی و مهندسی نرم‌افزار به شرح زیر توجیه می‌گردد:



شکل ۱: تفکیک مفهومی موجودیت‌ها (Entities) و اشیاء مقداری (Value Objects)

۱.۳ موجودیت‌ها (Entities)

۱. **User:** دارای شناسه یکتا (`id`). وضعیت حساب، نشانی‌ها و مشخصات آن در طول زمان تغییر می‌کند.

۲. **Product:** دارای شناسه یکتا (`id`). موجودی انبار، قیمت، آمار فروش و وضعیت فعال بودن آن جهش‌پذیر است.

۳. **Cart:** سبد خرید مشتری که اقلام آن در طول نشست کاری افزوده، ویرایش یا حذف می‌شوند (Mutable Aggregate).

۴. **Order:** سند رسمی خرید با شناسه یکتا (`orderId`) که وضعیت چرخه عمر آن دگرگون می‌شود.

۲.۳ اشیاء مقداری (Value Objects)

۱. **Quantity:** تعداد گسسته اقلام با تضمین نامتغیر $value > 0$.

۲. **Money:** مدل‌سازی مالی دقیق با `BigDecimal` و واحد پولی جهت پیشگیری از خطای محاسبات اعشاری.

۳. **Address:** نشانی پستی تحویل سفارش به صورت تغییرناپذیر با کد پستی ده‌رقمی.

۴. **ContactInfo:** شیء مقداری تغییرناپذیر اطلاعات تماس شامل ایمیل و تلفن همراه معتبر.

۵. **CartItem:** سطر سبد خرید شامل کالا، قیمت و تعداد.

۶. **OrderItem:** تصویر پایدار و تاریخی اقلام در لحظه ثبت سفارش (Immutable Snapshot).

۷. **Review:** دیدگاه و امتیاز ثبت‌شده برای کالا با سقف ۲۰۰۰ کاراکتر متن.

۸. **Coupon:** ضوابط و تخفیف کوپن.

۹. **Category:** دسته‌بندی کالا در کاتالوگ فروشگاه به همراه پرچم فعال بودن.

۱۰. **PaymentTransaction:** سند ثبت تراکنش مالی پرداخت سفارش.

۱۱. **انواع شمارشی (Enums):** UserRole، OrderStatus، PaymentMethod، PaymentStatus.

۴ کاتالوگ جامع کلاس‌های لایه مدل (Domain Model Catalog)

در این بخش، کلیه اجزای کلاس‌های لایه Model به طور کامل و منطبق بر اصل اتحاد مستند می‌شوند.

۱.۴ کلاس Quantity (Value Object — Record)

هدف: مدل‌سازی ریاضی و کسب‌وکاری تعداد اقلام در سبد، سفارش و انبار.
فیلدها:

- `private final int value`: مقدار عددی صحیح مثبت.

سازنده:

- `public Quantity(int value)`: سازنده فشرده رکورد با اعتبارسنجی نامتغیر.

متدها:

- `public int value()`: دریافت مقدار عددی (Observer – تولید خودکار توسط کامپایلر رکورد).
- `public Quantity add(Quantity other)`: تولید تعداد جدید حاصل جمع (Producer).
- `public Quantity subtract(Quantity other)`: تولید تعداد جدید حاصل تفریق (Producer).
- `public boolean isGreaterThan(Quantity other)`: مقایسه بزرگی (Observer).

نامتغیر نمایش (RI): `value > 0`.
تابع انتزاع (AF): `AF(q) = an exact strictly positive integer count equal to q.value`.

قرارداد متدها (Specification Contract)

```
Constructor:
   REQUIRES: value > 0
   MODIFIES: this
   EFFECTS: Initializes Quantity; throws IllegalArgumentException if value <= 0.
---
add(Quantity other):
   REQUIRES: other != null
   MODIFIES: nothing
   EFFECTS:    Returns new Quantity(this.value + other.value); throws NullPointerException if
null.
---
subtract(Quantity other):
   REQUIRES: other != null and this.value > other.value
   MODIFIES: nothing
   EFFECTS:   Returns new Quantity(this.value - other.value); throws IllegalArgumentException
if other.value >= this.value.
```

۲.۴ کلاس Money (Value Object — Record)

هدف: مدل‌سازی مالی دقیق جهت قیمت‌ها، مالیات، هزینه ارسال و تخفیف‌ها.
فیلدها:

- `private final BigDecimal amount`: مبلغ عددی با مقیاس استاندارد ۲ رقم اعشار.

- `private final String currency`: **واحد پولی استاندارد (مانند `"IRR"` یا `"TOMAN"`).**

ثابت‌ها:

- `public static final Money ZERO = new Money(BigDecimal.ZERO, "IRR");`

سازنده:

- `public Money(BigDecimal amount, String currency)`: **سازنده فشرده رکورد.**

متدها:

- `public BigDecimal amount()`: **دریافت مبلغ** (Observer).
- `public String currency()`: **دریافت واحد پول** (Observer).
- `public Money add(Money other)`: **جمع دو مبلغ هم‌واحد** (Producer).
- `public Money subtract(Money other)`: **تفریق دو مبلغ هم‌واحد** (Producer).
- `public Money multiply(Quantity quantity)`: **ضرب مبلغ در تعداد** (Producer).
- `public boolean isLessThan(Money other)`: **مقایسه کمتر بودن مبالغ** (Observer).
- `public boolean isNegative()`: **بررسی منفی بودن مبلغ** (Observer).

نامتغیر نمایش (RI): `amount != null && currency != null && !currency.isBlank() && amount.scale() == 2 && amount.signum() >= 0`.

تابع انتزاع (AF): `AF(m) = a monetary value of m.amount in currency m.currency`.

۳.۴ کلاس Address (Value Object — Record)

هدف: نگهداری مشخصات نشانی تحویل کالا مطابق با US 07 و قوانین پستی کشور.

فیلدها و مؤلفه‌های رکورد:

- `private final String addressId`: **شناسه محلی نشانی (مانند `"ADDR-1"`).**
- `private final String recipientName`: **نام گیرنده تحویل‌گیرنده.**
- `private final String phone`: **شماره تماس همراه گیرنده.**
- `private final String province`: **استان مقصد.**
- `private final String city`: **شهر مقصد.**
- `private final String streetAddress`: **نشانی پستی دقیق (خیابان، پلاک، واحد).**
- `private final String postalCode`: **کد پستی ده‌رقمی معتبر مطابق با الگوی** `^[0-9]{10}$`.
- `private final String title`: **عنوان نشانی (نظیر منزل، شرکت).**
- `private final boolean isDefault`: **پرچم پیش‌فرض بودن نشانی.**

متدها:

- **اکسسورهای رکورد:** `addressId()`، `recipientName()`، `phone()`، `province()`، `city()`، `streetAddress()`، `postalCode()`، `title()`، `isDefault()`.
- `public Address withDefault(boolean isDefault)`: **نسخه جدید با پرچم به‌روزرسانی‌شده** (Producer).
- `public String fullAddressText()`: **قالب‌بندی متنی کامل آدرس جهت درج روی فاکتور** (Observer).

۴.۴ کلاس User (Entity)

هدف: هویت و حساب کاربری خریداران و مدیران سامانه (US 01 تا US 07).
فیلدها:

- `private final String id`: **شناسه یکتا و تغییرناپذیر کاربر.**
- `private String fullName`: **نام و نام خانوادگی.**
- `private String email`: **آدرس ایمیل منحصربه‌فرد.**
- `private String phone`: **شماره تلفن همراه معتبر.**
- `private ContactInfo contactInfo`: **شیء مقداری کانال‌های ارتباطی کاربر.**
- `private String passwordHash`: **هش امنیتی رمز عبور** (SHA-256).
- `private UserRole role`: **نقش کاربری** (CUSTOMER, ADMIN, STOREKEEPER).
- `private boolean active`: **وضعیت فعال یا غیرفعال بودن حساب.**
- `private final LocalDateTime createdAt`: **زمان ایجاد حساب.**
- `private final List<Address> addresses`: **فهرست نشانی‌های ثبت‌شده.**

متدها:

- `public String getId()`، `public String getFullName()`، `public String getEmail()`، `getPhone()`، `public ContactInfo getContactInfo()`، `public UserRole getRole()`، `public boolean isActive()`، `public LocalDateTime getCreatedAt()` (Observers).
- `public void activate()`، `public void deactivate()` (Mutators).
- `public void updateProfile(String newFullName, String newPhone)` (Mutator).
- `public void changePassword(String newPasswordHash)` (Mutator).
- `public void addAddress(Address address)`، `public void updateAddress(Address updatedAddress)`، `public void removeAddress(String addressId)`، `public void setDefaultAddress(String addressId)` (Mutators).
- `public Optional<Address> getDefaultAddress()` (Observer).
- `public List<Address> getAddresses()`: **دریافت کپی غیرقابل تغییر نشانی‌ها** (Observer – **محافظت با** `List.copyOf`).

۵.۴ کلاس Product (Entity)

هدف: مدل‌سازی مشخصات کالا در کاتالوگ فروشگاه آنلاین (US 08–14, 33–36).
فیلدها:

- `private final String id`: **شناسه یکتای محصول.**
- `private String name`: **عنوان تجاری کالا.**

- `private String description`: **شرح تفصیلی کالا.**
- `private Money price`: **قیمت فعلی کالا.**
- `private int availableStock`: **تعداد موجودی فیزیکی در انبار** ($availableStock \geq 0$).
- `private String categoryId`: **شناسه دسته‌بندی والد.**
- `private boolean active`: **پرچم فعال بودن کالا در ویترین.**
- `private double averageRating`: **میانگین امتیاز نظرات (۰٫۰ تا ۵٫۰).**
- `private int reviewCount`: **تعداد کل دیدگاه‌های ثبت‌شده.**
- `private final LocalDateTime createdAt`: **زمان ایجاد و ورود کالا به سامانه.**
- `private int soldCount`: **شمارنده تجمیعی تعداد اقلام فروخته‌شده در پرداخت‌های موفق.**
- `private List<String> imageUrls`: **فهرست آدرس‌های اینترنتی تصاویر کالا.**

متدها:

- **اکسسورها:** `getId()`، `getName()`، `getDescription()`، `getPrice()`، `getAvailableStock()`، `getCategoryId()`، `isActive()`، `getAverageRating()`، `getReviewCount()`، `getCreatedAt()`، `getSoldCount()`، `getImageUrls()`.
- `public boolean isAvailable(Quantity required)`: **بررسی کفایت موجودی** (Observer).
- `public void updateDetails(String name, String desc, Money price, String categoryId)` (Mutator).
- `public void setActive(boolean active)`: **تغییر وضعیت انتشار توسط مدیر** (Mutator).
- `public void deductStock(Quantity quantity)`: **کسر قطعی موجودی کالا** (Mutator).
- `public void restock(Quantity quantity)`: **شارژ موجودی انبار** (Mutator).
- `public void recordSale(Quantity quantity)`: **ثبت فروش و افزایش** `soldCount` (Mutator).
- `public void setImageUrls(List<String> imageUrls)`: **به‌روزرسانی لینک تصاویر** (Mutator).
- `public void updateRatingStats(int newRatingScore)`: **محاسبه مجدد میانگین امتیاز** (Mutator).

۶.۴ کلاس CartItem (Value Object — Record)

هدف: مدل‌سازی هر سطر در سبد خرید مشتری (US 15–19).
فیلدها و مؤلفه‌ها: `productId: String`، `productName: String`، `unitPrice: Money`، `quantity: Quantity`.
متدها:

- **اکسسورهای رکورد:** `productId()`، `productName()`، `unitPrice()`، `quantity()`.
- `public Money getLineTotal()`: **محاسبه قیمت کل سطر حاصل ضرب قیمت در تعداد** (Observer).
- `public CartItem withQuantity(Quantity newQuantity)`: **تولید نسخه جدید با تعداد تغییریافته** (Producer).

۷.۴ کلاس Cart (Entity / Aggregate Root)

هدف: نگهداری سبد خرید مشتری به صورت ساختار جهش‌پذیر در طول نشست کاری.
فیلدها:

- `private final String cartId`: **شناسه سبد خرید (همنام با شناسه کاربر).**
- `private final String userId`: **شناسه کاربر مالک.**
- `private final Map<String, CartItem> items`: **اقلام سبد خرید بر مبنای شناسه کالا.**

متدها:

- **اکسسورها:** `getCartId()`، `getUserId()`.
- `public void addProduct(Product product, Quantity quantity)`: **افزودن یا افزایش تعداد کالا** (Mutator).
- `public void updateQuantity(String productId, Quantity newQuantity)`: **اصلاح تعداد** (Mutator).
- `public void removeProduct(String productId)`: **حذف سطر کالا** (Mutator).
- `public void removeCheckedOutItems(List<CartItem> checkedOutItems)`: **حذف انتخابی اقلام پرداخت‌شده** (Mutator).
- `public void clear()`: **تخلیه کامل سبد خرید** (Mutator).
- `public int quantityOf(String productId)`: **تعداد کالای موجود در سبد** (Observer).
- `public Money calculateSubtotal()`: **جمع مبالغ اقلام سبد خرید** (Observer).
- `public boolean isEmpty()`: **بررسی خالی بودن سبد** (Observer).
- `public List<CartItem> getItems()`: **بازگرداندن کپی تغییرناپذیر سطرهای سبد** (Observer).

۸.۴ کلاس OrderItem (Value Object — Record)

هدف: ثبت تغییرناپذیر اقلام در لحظه نهایی شدن سفارش (Snapshot).
فیلدها و مؤلفه‌ها: `productId: String`، `productName: String`، `unitPrice: Money`، `quantity: Quantity`، `lineTotal: Money`.
متدها: اکسسورهای رکورد: `productId()`، `productName()`، `unitPrice()`، `quantity()`، `lineTotal()`.
نامتغیر نمایش (RI): `lineTotal.equals(unitPrice.multiply(quantity))`.

۹.۴ کلاس Order (Entity / Aggregate Root)

هدف: سند رسمی سفارش مشتری به همراه کلیه اقلام، هزینه‌ها، نشانی و وضعیت چرخه حیات.
فیلدها:

- `private final String orderId`: **شناسه یکتای سفارش.**
- `private final String userId`: **شناسه مشتری ثبت‌کننده.**
- `private final List<OrderItem> items`: **سطرهای قطعی اقلام سفارش.**
- `private final Address deliveryAddress`: **نشانی تحویل سفارش.**

- `private final Money itemsSubtotal`: **مجموع قیمت اقلام.**
- `private final Money discountAmount`: **مبلغ تخفیف اعمال‌شده کوپن.**
- `private final Money shippingFee`: **هزینه حمل و نقل.**
- `private final Money payableAmount`: **مبلغ نهایی قابل پرداخت.**
- `private OrderStatus status`: **وضعیت چرخه عمر سفارش.**
- `private final LocalDateTime createdAt`: **تاریخ و ساعت ثبت سفارش.**
- `private LocalDateTime paidAt`: **تاریخ و ساعت پرداخت موفق.**

متدها:

- **اکسسورها:** `getOrderId()`، `getUserId()`، `getStatus()`، `getItemsSubtotal()`، `getDiscountAmount()`، `getShippingFee()`، `getPayableAmount()`، `getItems()`، `getDeliveryAddress()`، `getCreatedAt()`، `getPaidAt()`.
- `public void markAsPaid(LocalDateTime paymentTime)`: **ثبت وضعیت پرداخت موفق** (Mutator).
- `public void markAsPaymentFailed()`: **ثبت شکست در پرداخت با حفظ قابلیت تلاش دوباره** (Mutator).
- `public void updateStatus(OrderStatus newStatus)`: **تغییر وضعیت توسط مدیر یا انباردار** (Mutator).
- `public void cancel()`: **لغو سفارش** (Mutator).
- `public boolean canBeCancelled()`: **بررسی امکان‌پذیری لغو سفارش** (Observer).

۱۰.۴ کلاس PaymentTransaction (Value Object — Record)

هدف: ثبت قطعی تراکنش مالی و پرداخت سفارش (DS 08, US 24, US 25).
فیلدها و مؤلفه‌ها:

- `private final String transactionId`: **شناسه پیگیری یکتای تراکنش.**
- `private final String orderId`: **شناسه سفارش مرتبط.**
- `private final Money amount`: **مبلغ پرداختی.**
- `private final PaymentMethod method`: **روش پرداخت انتخابی.**
- `private final PaymentStatus status`: **نتیجه پردازش تراکنش.**
- `private final LocalDateTime timestamp`: **زمان دقیق انجام تراکنش.**
- `private final String referenceCode`: **کد ارجاع درگاه یا بانک.**

متدها: اکسسورهای رکورد: `transactionId()`، `orderId()`، `amount()`، `method()`، `status()`، `timestamp()`، `referenceCode()`.
نامتغیر نمایش (RI): `transactionId != null && orderId != null && amount != null && status != null`.

۱۱.۴ کلاس Category (Value Object / Entity)

هدف: دسته‌بندی موضوعی کالاها در ساختار کاتالوگ فروشگاه آنلاین.
فیلدها: `id: String`، `name: String`، `description: String`، `active: boolean`.
متدها: اکسسورهای استاندارد خواندنی `id()`، `name()`، `description()`، `active()`.

۱۲.۴ کلاس Coupon (Value Object — Record)

هدف: اعمال تخفیف‌های درصدی یا سقفی در مرحله تسویه‌حساب سفارش (US 22, DS 07).
فیلدها: `code: String`، `discountPercent: int`، `maxDiscount: Money`، `minOrderAmount: Money`، `expiryDate: LocalDate`.
متدها:

- `public Money calculateDiscount(Money subtotal)`: **محاسبه مبلغ تخفیف با رعایت سقف مجاز** (Observer).
- `public boolean isValid(Money subtotal, LocalDate currentDate)`: **اعتبارسنجی انقضا و حداقل خرید** (Observer).

۱۳.۴ کلاس Review (Value Object — Record)

هدف: ثبت نظر و امتیاز کیفی خریداران درباره محصولات خریداری‌شده (US 30–32, DS 10).
فیلدها و مؤلفه‌ها: `reviewId: String`، `userId: String`، `authorName: String`، `productId: String`، `rating: int (1..5)`، `commentText: String`، `createdAt: LocalDateTime`.
متدها: اکسسورهای رکورد: `reviewId()`، `userId()`، `authorName()`، `productId()`، `rating()`، `commentText()`، `createdAt()`.
نامتغیر نمایش (RI): `rating >= 1 && rating <= 5 && commentText != null && !commentText.isBlank() && commentText.length() <= 2000`.

۱۴.۴ کلاس ContactInfo (Value Object — Record)

هدف: کپسوله‌سازی تغییرناپذیر اطلاعات تماس کاربر (ایمیل و شماره تلفن همراه) با اعتبارسنجی عبارات باقاعده.
فیلدها و مؤلفه‌ها:

- `private final String email`: **آدرس ایمیل استاندارد مطابق با الگوی RFC**.
- `private final String phone`: **شماره تلفن همراه ۱۱ رقمی ایران شروع با 09**.

سازنده:

- `public ContactInfo(String email, String phone)`: **اعتبارسنجی حضور حداقل یک مشخصه ارتباطی و انطباق مقادیر با الگوهای اعتبارسنجی**.

متدها:

- `public String email()`، `public String phone()`: **دریافت مقادیر مؤلفه‌ها**.
- `public boolean hasEmail()`، `public boolean hasPhone()`: **بررسی حضور هر مؤلفه** (Observers).

نامتغیر نمایش (RI): حداقل یکی از فیلدهای `email` یا `phone` مقدار غیرتهی داشته و مقادیر با عبارات باقاعده مطابقت دارند.
تابع انتزاع (AF): `AF(c) = a contact channel pair with email c.email and phone c.phone`.

قرارداد متدها (Specification Contract)

```
Constructor(String email, String phone):
   REQUIRES: at least one of email or phone is non-null/non-blank, and valid format
   MODIFIES: this
   EFFECTS:  Validates RFC email and Iranian mobile patterns; throws IllegalArgumentException
if both missing or invalid.
```

۱۵.۴ انواع شمارشی مدل دامنه (Domain Enumerations)

۱. **UserRole: نقش‌های سطح دسترسی سامانه:** CUSTOMER, ADMIN, STOREKEEPER.

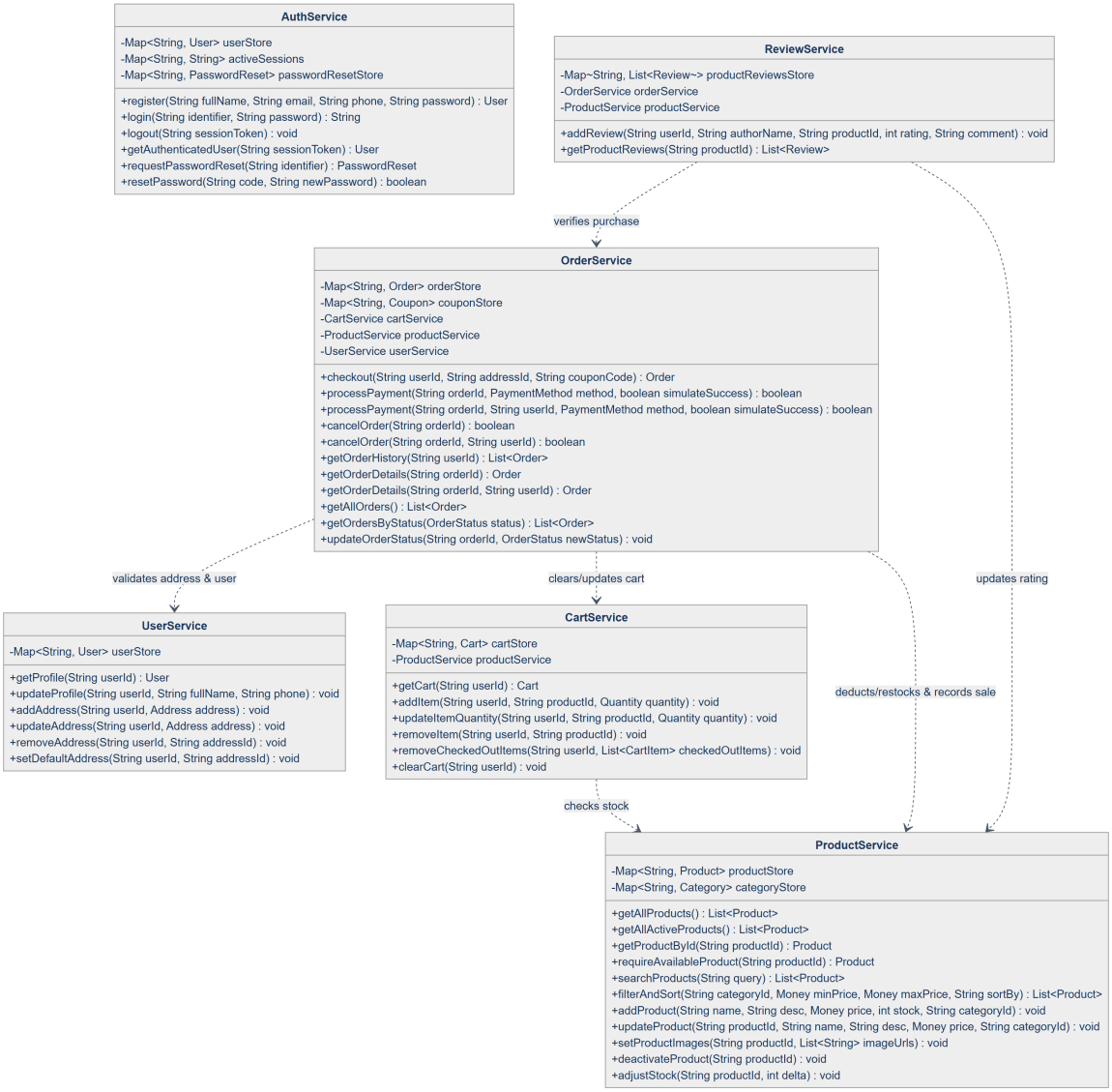
۲. **OrderStatus: مراحل چرخه حیات سفارش:** PENDING, PAYMENT_FAILED, PAID, PROCESSING, SHIPPED, DELIVERED, CANCELLED.

۳. **PaymentMethod: روش‌های پرداخت مجاز:** ONLINE_GATEWAY, WALLET, CASH_ON_DELIVERY.

۴. **PaymentStatus: وضعیت نتیجه تراکنش پرداخت:** SUCCESS, FAILED, CANCELLED.

۵ کاتالوگ جامع کلاس‌های لایه سرویس (Application & Domain Services)

بر اساس اسلاید ۴ سند پروژه، لایه Service هماهنگ‌کننده سناریوهای Use Case بین چند موجودیت است و وظیفه مدیریت داده‌های درون‌حافظه‌ای (In-Memory Storage) را بدون نیاز به لایه دیتابیس خارجی بر عهده دارد.



شکل ۲: کلاس‌های لایه سرویس و ارتباطات هماهنگ‌سازی آنها

۱.۵ AuthService

مسئولیت: مدیریت ثبت‌نام، نشست‌های کاری، احراز هویت و بازیابی رمز عبور شبیه‌سازی‌شده (DS 01, DS 02, DS 03).

فیلدهای ذخیره‌سازی:

- `private final Map<String, User> userStore`: **نگاشت ایمیل/شناسه به شیء کاربر.**
- `private final Map<String, String> activeSessions`: **نگاشت توکن نشست به شناسه کاربر.**

- `private final Map<String, PasswordReset> passwordResetStore`: **نگاشت توکن بازیابی رمز عبور.**

قرارداد متدها (Specification Contract)

```
register(String fullName, String email, String phone, String password):
   REQUIRES: valid email or phone, unique across system, password length >= 6.
   MODIFIES: userStore
   EFFECTS:      Creates User with hashed password, stores in userStore, returns User. Throws
DuplicateUserException if exists.
---
login(String identifier, String password):
   REQUIRES: identifier != null, password != null
   MODIFIES: activeSessions
   EFFECTS:                Authenticates via email or phone; generates session token; throws
AuthenticationException if invalid.
---
requestPasswordReset(String identifier):
   REQUIRES: user with identifier exists
   MODIFIES: passwordResetStore
   EFFECTS: Generates 6-digit verification code valid for 10 minutes.
---
resetPassword(String code, String newPassword):
   REQUIRES: code is valid and non-expired
   MODIFIES: userStore, activeSessions, passwordResetStore
   EFFECTS:        Updates password hash, removes reset code, invalidates all existing active
sessions of user.
```

۲.۵ UserService

مسئولیت: مدیریت اطلاعات پروفایل مشتری و دفترچه نشانی‌ها (DS 04, US 05, US 06, US 07).
فیلدهای ذخیره‌سازی: `private final Map<String, User> userStore`.

قرارداد متدها (Specification Contract)

```
updateProfile(String userId, String fullName, String phone):
   REQUIRES: user exists, fullName non-blank, phone unique across users
   MODIFIES: userStore
   EFFECTS:     Updates profile data; throws DuplicateUserException if phone taken by another
user.
---
updateAddress(String userId, Address address):
   REQUIRES: user exists, address != null and addressId exists
   MODIFIES: userStore
   EFFECTS: Replaces existing address in user's address book.
```

۳.۵ ProductService

مسئولیت: مدیریت کاتالوگ محصولات، جستجو، فیلتر، مرتب‌سازی و انبارداری (DS 05, DS 11, DS 12).
فیلدهای ذخیره‌سازی:

- `private final Map<String, Product> productStore`: **نگاشت شناسه کالا به شیء محصول.**

- `private final Map<String, Category> categoryStore`: **نگاشت دسته‌بندی‌ها.**

قرارداد متدها (Specification Contract)

```
searchProducts(String query):
   REQUIRES: query != null and length between 3 and 100 characters
   MODIFIES: nothing
   EFFECTS: Returns active products in active categories matching query (case-insensitive).
---
filterAndSort(String catId, Money min, Money max, String sort):
   REQUIRES: sort in {"price_asc", "price_desc", "rating", "newest", "popular"}
   MODIFIES: nothing
   EFFECTS: Filters by category and price range, then sorts accordingly.
```

۴.۵ CartService

مسئولیت: هماهنگی سبد خرید مشتری با کاتالوگ محصولات (DS 06).
فیلدهای ذخیره‌سازی:

- `private final Map<String, Cart> cartStore`: **نگاشت شناسه کاربر به سبد خرید.**
- `private final ProductService productService`: **سرویس کاتالوگ جهت بررسی موجودی.**

قرارداد متدها (Specification Contract)

```
removeCheckedOutItems(String userId, List<CartItem> checkedOutItems):
   REQUIRES: userId != null, checkedOutItems != null
   MODIFIES: cartStore
   EFFECTS:      Removes from Cart only those items that were checked out and unchanged since
checkout.
```

۵.۵ OrderService

مسئولیت: تسویه‌حساب، اعمال کوپن، پرداخت، ثبت فاکتور و لغو سفارش (DS 07, DS 08, DS 09, DS 12).
فیلدهای ذخیره‌سازی:

- `private final Map<String, Order> orderStore`: **نگاشت شناسه سفارش به شیء سفارش.**
- `private final Map<String, Coupon> couponStore`: **مخزن کدهای تخفیف معتبر.**
- `private final CartService cartService`: **هماهنگی با سبد خرید.**
- `private final ProductService productService`: **هماهنگی با انبار کالاها.**

قرارداد متدها (Specification Contract)

```
processPayment(String orderId, String userId, PaymentMethod method, boolean
simulateSuccess):
   REQUIRES: order belongs to userId and status is PENDING or PAYMENT_FAILED
   MODIFIES: orderStore, productService, cartService
   EFFECTS:     Validates stock; if success: deducts stock, increments soldCount, marks PAID,
cleans checked-out items from cart; if failure: marks PAYMENT_FAILED.
---
```

```
cancelOrder(String orderId, String userId):
   REQUIRES: order belongs to userId and canBeCancelled() is true
   MODIFIES: orderStore, productService
   EFFECTS: Cancels order; if previously PAID, restocks inventory in productService.
```

۶.۵ ReviewService

مسئولیت: ثبت و بازیابی نظرات و امتیازات محصولات با اعتبارسنجی خرید کاربر (DS 10, US 30–32).
فیلدهای ذخیره‌سازی:

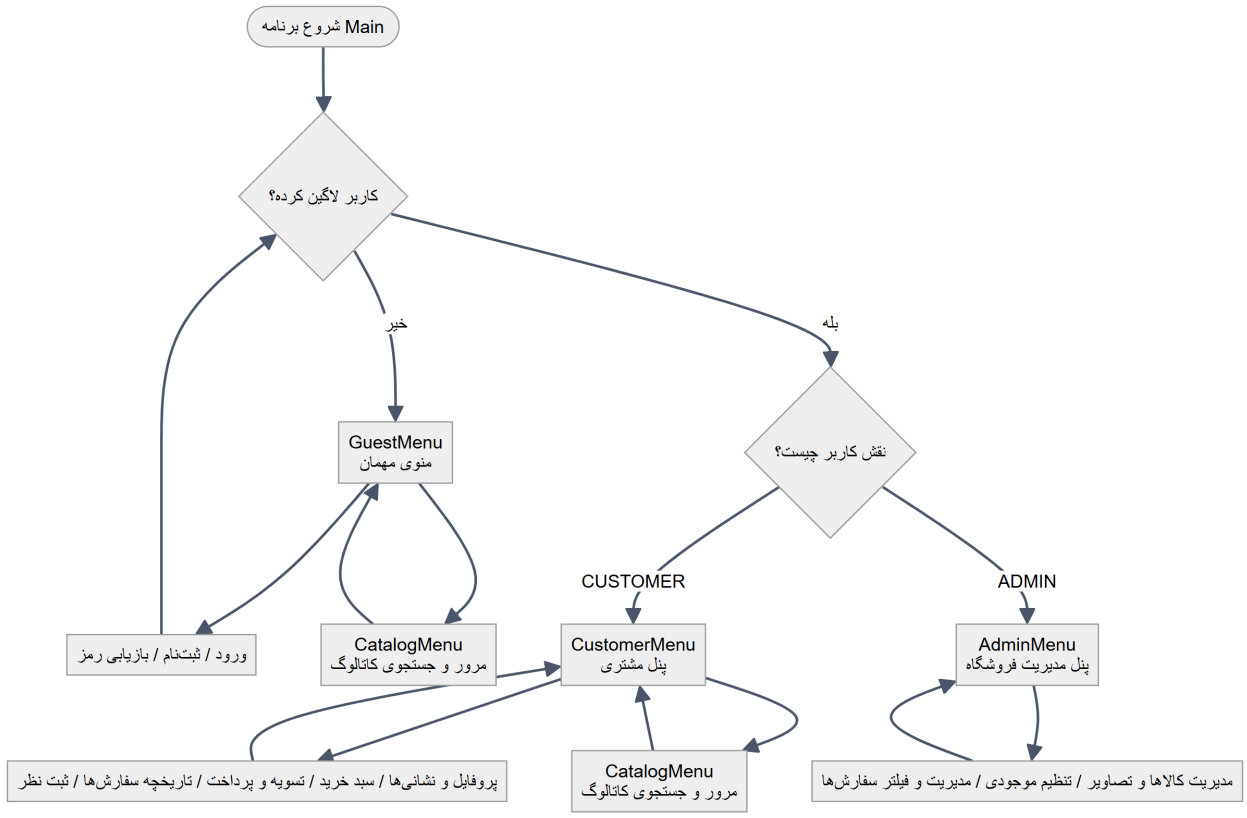
- `private final Map<String, List<Review>> productReviewsStore`: **نگاشت شناسه کالا به لیست نظرات.**
- `private final OrderService orderService`: **اعتبارسنجی پیش‌خرید کالا توسط کاربر.**

قرارداد متدها (Specification Contract)

```
addReview(String userId, String authorName, String productId, int rating, String comment):
   REQUIRES:    rating between 1 and 5, comment non-blank <= 2000 chars, userId has purchased product in PAID order
   MODIFIES: productReviewsStore, productService
   EFFECTS:          Creates Review, adds to store, updates product rating statistics; throws UnauthorizedActionException if not purchased.
```

۶ طراحی معماری لایه Presentation (CLI Layer Specification)

لایه واسط کاربری متنی (CLI) طبق اسلاید ۴ سند پروژه، تعامل کاربر از طریق ترمینال را بر عهده دارد. برای رعایت اصول Clean Code، واسط کاربری به منوهای ماژولار زیر تقسیم شده است:



شکل ۳: جریان اجرای منوهای لایه CLI و تعامل با سرویس‌ها

۱.۶ اجزای اصلی لایه CLI

۱. **ConsoleIO:** کلاس کمکی ورودی و خروجی استاندارد، دریافت امن رشته، عدد و رمز عبور با اعتبارسنجی در مرز ورودی.

۲. **MainMenuController:** کنترل‌کننده مرکزی هدایت بین منوی مهمان و منوهای نقش‌محور بر اساس توکن نشست فعال.

۳. **GuestMenu:** منوی کاربر پیش از احراز هویت شامل ثبت‌نام، ورود، بازیابی رمز و مرور کاتالوگ (US 01, 02, 04, 08–14).

۴. **CustomerMenu:** منوی تعاملی خریدار احرازهویت‌شده شامل پروفایل، نشانی‌ها، سبد، تسویه، پرداخت، پیگیری و لغو سفارش و ثبت نظر.

۵. **AdminMenu:** منوی پنل مدیریت سامانه با کنترل دسترسی ویژه نقش `ADMIN` جهت مدیریت کالاها، انبار و مشاهده و فیلتر کلیه سفارش‌ها.

۶. **CatalogMenu:** مؤلفه ماژولار و مشترک مرور و جستجوی کاتالوگ با امکان فیلتر بر اساس دسته‌بندی و بازه قیمت و مرتب‌سازی چندگانه.

۷ مشخصات کلاس راه‌انداز اصلی (Main.java) و ابزارهای پشتیبان

کلاس Main نقطه ورود به برنامه است (اسلاید ۴ سند پروژه). وظایف سه‌گانه آن عبارتند از:

۱. **نمونه‌سازی و تزریق وابستگی‌ها (Manual Dependency Injection):** ایجاد مخازن درون‌حافظه‌ای و اتصال لایه‌ها.

۲. **بارگذاری داده‌های اولیه آزمایشی (Data Seeding):** کلاس `DataSeeder` حساب‌های کاربری زیر را به صورت پیش‌فرض بارگذاری می‌نماید:

نقش کاربری	ایمیل شناسه	شماره همراه	رمز عبور پیش‌فرض	
مدیر سیستم (ADMIN)	`admin@shop.com`	—	`Admin@123`	
مشتری نمونه (CUSTOMER)	`customer@shop.com`	`09121112233`	`Pass@123`	

جدول ۳: مشخصات حساب‌های کاربری بارگذاری‌شده در زمان راه‌اندازی برنامه

۳. **ابزارهای کمکی امنیتی (PasswordUtil):** ارائه تابع هشینگ یک‌طرفه `hashPassword` بر مبنای الگوریتم استاندارد SHA-256.

۸ نمودارهای جامع ساختاری و روابط (UML Class Diagrams)

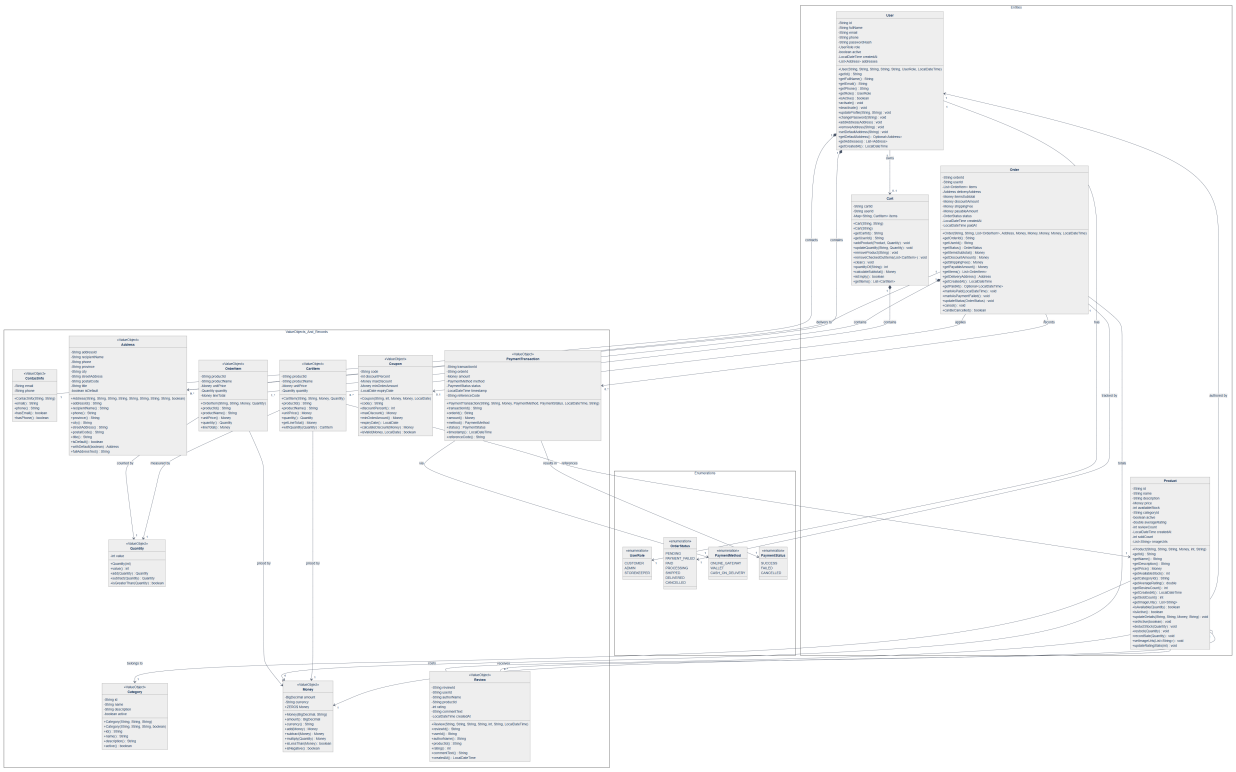
۱.۸ جدول جامع ارتباط‌ها و تعداد آن‌ها (Associations & Multiplicities)

کلاس مبدأ	کلاس مقصد	نوع و کاردینالیتی	شرح کسب‌وکاری رابطه
User	Address	$1 \rightarrow 0..*$	هر کاربر دارای صفر یا چند نشانی است؛ حداکثر یک پیش‌فرض.
User	ContactInfo	$1 \rightarrow 1$	اطلاعات تماس شامل ایمیل و موبایل کاربر.
User	Cart	$1 \rightarrow 0..1$	سبد خرید در اولین استفاده ساخته می‌شود؛ مالکیت ثابت است.
User	Order	$1 \rightarrow 0..*$	اتصال با شناسه `userId` برای تاریخچه سفارش‌ها.
Cart	CartItem	$1 \rightarrow 0..*$	برای هر محصول حداکثر یک قلم در سبد وجود دارد.
CartItem	Product	$* \rightarrow 1$	نگهداری شناسه، نام و قیمت در لحظه افزودن به سبد.
Product	Category	$* \rightarrow 1$	دسته‌بندی کالا؛ دسته‌بندی والد باید فعال باشد.
Order	OrderItem	$1 \rightarrow 1..*$	اقلام سفارش پس از تسویه قطعی و تغییرناپذیرند.
Order	PaymentTransaction	$1 \rightarrow 0..*$	ثبت کلیه تراکنش‌های موفق و ناموفق پرداخت سفارش.
Review	User / Product	$* \rightarrow 1$	ثبت نظر توسط خریدار پیشین کالا در سفارش پرداخت‌شده.

جدول ۴: جدول جامع ارتباطات، نوع رابطه‌ها و تعداد در لایه مدل

۲.۸ نمودار روابط کلاس‌های لایه Model (Core Domain Model)

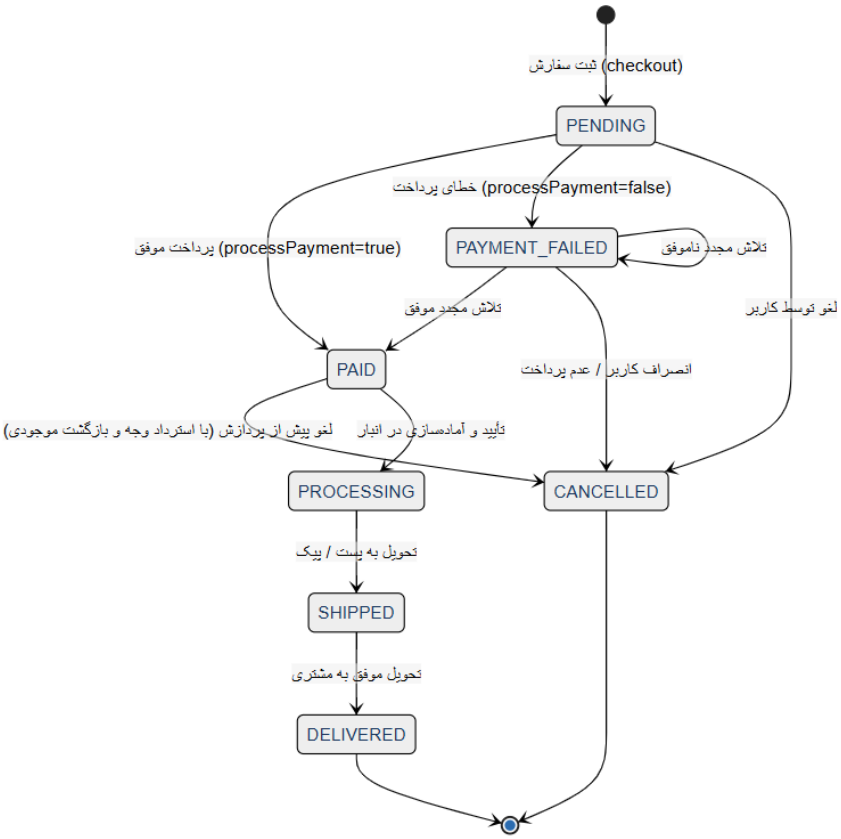
نمودار شکل ۴ کلیه کلاس‌های لایه Model، ویژگی‌ها، متدها، جهت روابط، چندگانگی‌ها و انواع شمارشی را بر مبنای اصل اتحاد کامل نمایش می‌دهد:



شکل ۴: نمودار جامع کلاس‌های لایه مدل دامنه (Core Domain UML Class Diagram)

۳.۸ نمودار ماشین وضعیت چرخه عمر سفارش (Order Lifecycle Statechart)

نمودار شکل ۵ گذارهای مجاز وضعیت سفارش (`OrderStatus`) از زمان تسویه‌حساب تا تحویل، شکست پرداخت، تلاش دوباره یا لغو را بر اساس اسلاید ۳ سند پروژه نمایش می‌دهد:



شکل ۵: نمودار ماشین وضعیت چرخه حیات سفارش (Order Lifecycle Statechart)

۹ اعتبارسنجی جامع مدل و ارزیابی دەگانه (Validation & Compliance)

مطابق چک‌لیست اعتبارسنجی، تمامی ۱۰ معیار ارزیابی مدل در جدول ۵ بررسی و تأیید قطعی شدند:

ردیف	موضوع ارزیابی	نحوه انطباق در این مدل	وضعیت
۱	**پوشش ۱۰۰٪ داستان‌های کاربری**	هر ۳۶ یوزر استوری در ماتریس ردیابی نگاشت شده و متد مشخص دارند.	**تأیید شد**
۲	**پوشش ۱۲ داستان دامنه**	فرآیندهای ۱۲ داستان دامنه مستقیماً به جریان فراخوانی بین لایه‌ها تبدیل شدند.	**تأیید شد**
۳	**تفکیک قطعی Entity و VO**	موجودیت‌ها دارای شناسه یکتا و چرخه عمر متغیرند؛ اشیاء مقداری بدون شناسه و تغییرناپذیرند.	**تأیید شد**
۴	**چندگانگی و جهت روابط**	جهت تمام یال‌ها و کاردینالیتی بر روی دیاگرام و جداول مستند شده است.	**تأیید شد**
۵	**امضای کامل متدها با نوع داده**	تمامی متدها با پارامترهای دارای نوع صریح و نوع بازگشتی معین مستند شدند.	**تأیید شد**
۶	**استقلال مطلق لایه Model**	کلاس‌های Model هیچ ارجاعی به لایه‌های بالاتر ندارند.	**تأیید شد**
۷	**عدم تعریف لایه دیتابیس مجزا**	ساختارهای Map درون سرویس‌ها کپسوله شدند.	**تأیید شد**
۸	**رعایت استانداردهای Java**	پکیج‌ها با حروف کوچک، کلاس‌ها با PascalCase و متدها با camelCase.	**تأیید شد**
۹	**رعایت اصول Clean Code**	رعایت اصل تک‌مسئولیتی، تفکیک فرمان و پرسش، و متدهای با اندازه کنترل‌شده.	**تأیید شد**
۱۰	**مستندسازی قراردادها و نامتغیرها**	درج پیش‌شرط، پس‌شرط، RI و AF برای انواع داده کلیدی.	**تأیید شد**

جدول ۵: جدول ارزیابی و انطباق دەگانه مدل با الزامات آموزشی و استانداردهای طراحی

۱۰ ثبت وضعیت و آمادگی برای ورود به فاز بعدی

با تکمیل این سند، مدل‌سازی دامنه و تدوین ساختار کلاس‌ها به طور کامل، دقیق و یکپارچه بر مبنای اصل اتحاد دیاگرام و متن تثبیت گردید. این سند به عنوان خط مبنای قطعی و مصوب پیاده‌سازی عمل خواهد کرد.